🛡️ Audit Sécurité InvoiceAI — Production Ready

> **Confidentiel — Business Project Flow · 2026**
> Stack : React / TypeScript / Supabase / Edge Functions / Vercel
> Approche : Security by Design — Paranoid Mode

1️⃣ Score global de sécurité

Domaine	Score	Statut
Authentification	55/100	🟠 Incomplet
Isolation multi-tenant (RLS)	40/100	🔴 Critique
Sécurité API / Secrets	45/100	🔴 Critique
Frontend (XSS, CSP)	50/100	🟠 Faible
Uploads / PDF	35/100	🔴 Critique
RGPD / Conformité	30/100	🔴 Non conforme
Infra / Déploiement	70/100	🟢 Acceptable
Logs / Audit trail	10/100	🔴 Inexistant
Protection abus	20/100	🔴 Inexistant

Score global estimé : 42/100 — Non production-ready

> Ce score est typique d'un projet sorti d'un builder IA (Lovable). L'architecture est bonne, l'implémentation sécurité manque structurellement. Tout est corrigeable en 2–3 semaines.

2️⃣ Top 10 vulnérabilités critiques

#1 — Absence de RLS sur les tables Supabase (IDOR massif)
Sans Row Level Security, tout utilisateur authentifié peut lire/modifier les factures, clients et données TVA de tous les autres utilisateurs via l'API Supabase directe. Criticité : maximale.

#2 — Clés API tierces exposées côté client
Si `VITE_CLAUDE_API_KEY`, `VITE_BILLIT_KEY` ou toute autre clé est préfixée `VITE_`, elle est bundlée dans le JS client et lisible en clair dans les DevTools de n'importe quel navigateur.

#3 — Génération PDF sans sandbox (SSRF / Path Traversal)
React-PDF côté client est sûr. Mais si la génération passe par un service serverless qui construit du HTML/CSS avant de le convertir, un input utilisateur non sanitisé peut injecter des ressources externes (SSRF) ou traverser le filesystem.

#4 — Uploads non validés (Supabase Storage)
Sans vérification du type MIME réel (magic bytes), un attaquant peut uploader un fichier `.html` ou `.svg` renommé en `.pdf`, déclencher du XSS stocké accessible via l'URL publique du bucket.

#5 — Broken Access Control sur les endpoints Edge Functions
Si les Edge Functions ne vérifient pas que `user_id` du JWT correspond à la ressource demandée, un attaquant peut modifier l'ID

dans la requête pour accéder aux données d'un autre compte (IDOR).

#6 — Absence de rate limiting
Les endpoints d'auth (signup, reset password) et de génération IA sont sans limitation. Vecteurs : credential stuffing, déni de service économique (coût API Claude), énumération de comptes.

#7 — JWT mal configuré (expiration, rotation)
Par défaut Supabase émet des JWT longue durée. Sans rotation de refresh token et sans invalidation côté serveur, un token volé reste valide indéfiniment.

#8 — Pas de Content Security Policy
Sans CSP header, le navigateur accepte n'importe quelle source de script. Un XSS réussi peut exfiltrer les tokens JWT, les données de factures affichées, les clés stockées en localStorage.

#9 — Données sensibles en localStorage
Les JWT et données de session souvent mis en `localStorage` par défaut sont accessibles à tout script de la page (XSS). Doit être en `httpOnly cookie` ou mémoire uniquement.

#10 — Absence totale d'audit trail
Aucun log de qui a accédé à quelle facture, aucune traçabilité des modifications. En cas d'incident ou d'audit RGPD, tu n'as aucune preuve et aucune capacité d'investigation.

3️⃣ Audit par couche

🖥️ Frontend — React / TypeScript / Tailwind

XSS (Cross-Site Scripting)

React échappe les valeurs par défaut dans JSX, mais les patterns dangereux existent :

```tsx
// ❌ DANGEREUX — jamais ça
<div dangerouslySetInnerHTML={{ __html: invoice.description

// ❌ DANGEREUX — concatenation dans les URLs
<a href={`javascript:${userInput}`}>lien</a>

// ✅ Toujours sanitiser si HTML est nécessaire
import DOMPurify from 'dompurify';
<div dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(

// ✅ Pour les URLs, valider le schéma
const safeUrl = (url: string) => {
  try {
    const parsed = new URL(url);
    if (![‘http:’, ‘https:’].includes(parsed.protocol)) ret
    return url;
  } catch { return '#'; }
};
```

Content Security Policy — à configurer dans `vercel.json`

```json
{
  "headers": [
    {
      "source": "/(.*)",
      "headers": [
        {
          "key": "Content-Security-Policy",
          "value": "default-src 'self'; script-src 'self' '
        },
        { "key": "X-Frame-Options", "value": "DENY" },
        { "key": "X-Content-Type-Options", "value": "nosnif
        { "key": "Referrer-Policy", "value": "strict-origin
        { "key": "Permissions-Policy", "value": "camera=(),
        { "key": "Strict-Transport-Security", "value": "max
      ]
    }
  ]
}
```

Variables d'environnement — règle absolue

```bash
```

```
# ❌ JAMAIS ces variables dans le frontend
VITE_CLAUDE_API_KEY=sk-...
VITE_BILLIT_SECRET=...
VITE_STRIPE_SECRET=sk_live_...

# ✅ Seules ces variables sont acceptables côté client
VITE_SUPABASE_URL=https://xxx.supabase.co          # publique
VITE_SUPABASE_ANON_KEY=eyJ...                       # publiqu
VITE_STRIPE_PUBLIC_KEY=pk_live_...                  # publiqu
```

Toutes les autres clés vivent uniquement dans les Edge Functions Supabase.

Validation des formulaires côté client

```typescript
// Utiliser Zod pour validation stricte
import { z } from 'zod';

const InvoiceSchema = z.object({
  clientName: z.string().min(1).max(200).regex(/^[\w\s\-'.]
  amount: z.number().positive().max(999999.99),
  vatNumber: z.string().regex(/^(BE|FR)\d{10,11}$/).optiona
  description: z.string().max(2000),
});

// La validation côté client n'est QUE UX — la vraie valida
```

⚙️ Backend — Edge Functions / API Serverless

Pattern de base pour toute Edge Function — Auth + Autorisation + Validation

typescript

```typescript
// supabase/functions/_shared/auth.ts
import { createClient } from '@supabase/supabase-js';

export async function getAuthenticatedUser(req: Request) {
  const authHeader = req.headers.get('Authorization');
  if (!authHeader?.startsWith('Bearer ')) {
    throw new Response(JSON.stringify({ error: 'Unauthorize
  }

  const token = authHeader.split(' ')[1];
  const supabase = createClient(
    Deno.env.get('SUPABASE_URL')!,
    Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')! // Service r
  );

  const { data: { user }, error } = await supabase.auth.get
  if (error || !user) {
    throw new Response(JSON.stringify({ error: 'Invalid tok
  }
  return user;
}
```

typescript

```
// supabase/functions/generate-invoice/index.ts
import { getAuthenticatedUser } from '../_shared/auth.ts';

Deno.serve(async (req) => {
  // 1. CORS
  if (req.method === 'OPTIONS') {
    return new Response(null, {
      headers: {
        'Access-Control-Allow-Origin': Deno.env.get('ALLOWE
        'Access-Control-Allow-Methods': 'POST',
        'Access-Control-Allow-Headers': 'Authorization, Con
      }
    });
  }

  try {
    // 2. Auth obligatoire
    const user = await getAuthenticatedUser(req);

    // 3. Parse + validation stricte du body
    const body = await req.json();
    const parsed = InvoiceInputSchema.parse(body); // Zod c

    // 4. Vérification que la ressource appartient à l'util
    const { data: client, error } = await supabase
      .from('clients')
      .select('id')
      .eq('id', parsed.clientId)
      .eq('user_id', user.id)  // ← CRITIQUE : toujours fil
      .single();

    if (error || !client) {
```

```
      return new Response(JSON.stringify({ error: 'Resource
      // Ne pas dire "Forbidden" — ne pas confirmer l'exist
    }

    // 5. Logique métier...

  } catch (err) {
    if (err instanceof Response) return err;
    console.error(err); // Log serveur uniquement, jamais e
    return new Response(JSON.stringify({ error: 'Internal e
  }
});
```

Rate Limiting sur les Edge Functions

```typescript
```

```
// supabase/functions/_shared/rateLimit.ts
export async function checkRateLimit(
  supabase: SupabaseClient,
  userId: string,
  action: string,
  maxRequests: number,
  windowSeconds: number
): Promise<void> {
  const windowStart = new Date(Date.now() - windowSeconds *

  const { count } = await supabase
    .from('rate_limit_logs')
    .select('*', { count: 'exact', head: true })
    .eq('user_id', userId)
    .eq('action', action)
    .gte('created_at', windowStart);

  if ((count ?? 0) >= maxRequests) {
    throw new Response(
      JSON.stringify({ error: 'Too Many Requests', retryAft
      { status: 429, headers: { 'Retry-After': String(windo
    );
  }

  await supabase.from('rate_limit_logs').insert({ user_id:
}

// Usage
await checkRateLimit(supabase, user.id, 'generate_invoice',
await checkRateLimit(supabase, user.id, 'claude_api', 20, 3
```

Protection SSRF sur les appels externes

typescript

```typescript
const ALLOWED_EXTERNAL_DOMAINS = [
  'api.anthropic.com',
  'api.billit.be',
  'ec.europa.eu',        // VIES TVA
  'api.stripe.com',
  'api.resend.com',
];

function validateExternalUrl(url: string): void {
  const parsed = new URL(url);
  if (!ALLOWED_EXTERNAL_DOMAINS.some(d => parsed.hostname =
    throw new Error(`SSRF protection: domain ${parsed.hostn
  }
}
```

🗄️ Base de données — Supabase / PostgreSQL / RLS

Schema RLS complet pour InvoiceAI

sql

```sql
-- ============================================================
-- ACTIVATION RLS SUR TOUTES LES TABLES
-- ============================================================

ALTER TABLE companies        ENABLE ROW LEVEL SECURITY;
ALTER TABLE clients          ENABLE ROW LEVEL SECURITY;
ALTER TABLE invoices         ENABLE ROW LEVEL SECURITY;
ALTER TABLE invoice_lines    ENABLE ROW LEVEL SECURITY;
ALTER TABLE payments         ENABLE ROW LEVEL SECURITY;
ALTER TABLE audit_logs       ENABLE ROW LEVEL SECURITY;

-- ============================================================
-- TABLE: companies (multi-entreprises par utilisateur)
-- ============================================================

CREATE POLICY "users_own_companies"
ON companies FOR ALL
USING (auth.uid() = user_id)
WITH CHECK (auth.uid() = user_id);

-- ============================================================
-- TABLE: clients
-- ============================================================

CREATE POLICY "users_own_clients"
ON clients FOR ALL
USING (
  EXISTS (
    SELECT 1 FROM companies
    WHERE companies.id = clients.company_id
    AND companies.user_id = auth.uid()
  )
```

```sql
)
WITH CHECK (
  EXISTS (
    SELECT 1 FROM companies
    WHERE companies.id = clients.company_id
    AND companies.user_id = auth.uid()
  )
);

-- ============================================================
-- TABLE: invoices
-- ============================================================

CREATE POLICY "users_own_invoices"
ON invoices FOR ALL
USING (
  EXISTS (
    SELECT 1 FROM companies
    WHERE companies.id = invoices.company_id
    AND companies.user_id = auth.uid()
  )
)
WITH CHECK (
  EXISTS (
    SELECT 1 FROM companies
    WHERE companies.id = invoices.company_id
    AND companies.user_id = auth.uid()
  )
);

-- ============================================================
-- TABLE: invoice_lines
```

```sql
-- ============================================================

CREATE POLICY "users_own_invoice_lines"
ON invoice_lines FOR ALL
USING (
  EXISTS (
    SELECT 1 FROM invoices
    JOIN companies ON companies.id = invoices.company_id
    WHERE invoices.id = invoice_lines.invoice_id
    AND companies.user_id = auth.uid()
  )
);

-- ============================================================
-- AUDIT LOGS — lecture seule, insertion via service role
-- ============================================================

CREATE POLICY "users_read_own_audit_logs"
ON audit_logs FOR SELECT
USING (auth.uid() = user_id);

-- ============================================================
-- VÉRIFICATION — lister toutes les tables sans RLS
-- ============================================================
SELECT tablename
FROM pg_tables
WHERE schemaname = 'public'
AND tablename NOT IN (
  SELECT tablename FROM pg_policies
);
```

Protection SQL Injection

typescript

```typescript
// ❌ DANGEREUX — construction de query dynamique
const { data } = await supabase.rpc('search_clients', {
  query: `%${userInput}%`
});

// ✅ Toujours utiliser les méthodes du SDK
const { data } = await supabase
  .from('clients')
  .select('*')
  .ilike('name', `%${sanitize(userInput)}%`)
  .limit(50); // Toujours un LIMIT

// ✅ Pour les fonctions SQL custom — préférer SECURITY INV
CREATE OR REPLACE FUNCTION search_clients(search_term TEXT)
RETURNS SETOF clients
LANGUAGE sql
SECURITY INVOKER -- respecte RLS
AS $$
  SELECT * FROM clients
  WHERE name ILIKE '%' || search_term || '%'
  LIMIT 50;
$$;
```

Chiffrement des données sensibles (Supabase Vault)

sql

```sql
-- Activer Vault pour les données ultra-sensibles (IBAN, nu
-- Extensions → pgsodium + supabase_vault

SELECT vault.create_secret('billit_api_key', 'valeur-de-la-

CREATE OR REPLACE FUNCTION get_billit_credentials()
RETURNS TEXT
LANGUAGE sql
SECURITY DEFINER
AS $$
  SELECT decrypted_secret FROM vault.decrypted_secrets
  WHERE name = 'billit_api_key';
$$;
```

📦 Uploads & Génération PDF

Validation stricte des uploads (magic bytes)

```typescript
```

```
// supabase/functions/upload-document/index.ts

const ALLOWED_MIME_TYPES = {
  'application/pdf': [0x25, 0x50, 0x44, 0x46],      // %PDF
  'image/jpeg': [0xFF, 0xD8, 0xFF],
  'image/png': [0x89, 0x50, 0x4E, 0x47],
};

const MAX_FILE_SIZE = 5 * 1024 * 1024; // 5 MB

async function validateFile(file: ArrayBuffer, declaredType
  // 1. Taille maximale
  if (file.byteLength > MAX_FILE_SIZE) {
    throw new Error('File too large');
  }

  // 2. Type MIME dans la whitelist
  if (!(declaredType in ALLOWED_MIME_TYPES)) {
    throw new Error('File type not allowed');
  }

  // 3. Vérification des magic bytes
  const bytes = new Uint8Array(file.slice(0, 8));
  const expectedMagic = ALLOWED_MIME_TYPES[declaredType as
  const isValid = expectedMagic.every((byte, i) => bytes[i]

  if (!isValid) {
    throw new Error('File content does not match declared t
  }

  // 4. Scanner le PDF pour du JS embarqué
  if (declaredType === 'application/pdf') {
```

```
    const text = new TextDecoder().decode(file.slice(0, 102
    if (text.includes('/JavaScript') || text.includes('/JS
      throw new Error('PDF contains JavaScript — rejected')
    }
  }
}
```

Politique de bucket Supabase Storage

```sql
```

```sql
-- Bucket "invoices" — privé, accès uniquement via URL sign
INSERT INTO storage.buckets (id, name, public, file_size_li
VALUES (
  'invoices',
  'invoices',
  false,                                    -- JAMAIS publi
  5242880,                                  -- 5 MB
  ARRAY['application/pdf', 'image/jpeg', 'image/png']
);

-- Upload : uniquement le propriétaire
CREATE POLICY "owner_upload_invoice"
ON storage.objects FOR INSERT
WITH CHECK (
  bucket_id = 'invoices'
  AND (storage.foldername(name))[1] = auth.uid()::text
);

-- Lecture : uniquement le propriétaire
CREATE POLICY "owner_read_invoice"
ON storage.objects FOR SELECT
USING (
  bucket_id = 'invoices'
  AND (storage.foldername(name))[1] = auth.uid()::text
);

-- Suppression : uniquement le propriétaire
CREATE POLICY "owner_delete_invoice"
ON storage.objects FOR DELETE
USING (
  bucket_id = 'invoices'
```

```
  AND (storage.foldername(name))[1] = auth.uid()::text
);
```

```typescript
// URL signée temporaire — jamais d'URL publique pour des f
const { data } = await supabase.storage
  .from('invoices')
  .createSignedUrl(`${userId}/${invoiceId}.pdf`, 3600); //
```

🔑 Authentification

Configuration Supabase Auth durcie

```toml
```

```
# supabase/config.toml
[auth]
site_url = "https://invoiceai.be"
additional_redirect_urls = []                # Whitelist stric
jwt_expiry = 3600                            # 1h
enable_refresh_token_rotation = true         # ← ACTIVER
refresh_token_reuse_interval = 10

[auth.email]
enable_signup = true
enable_confirmations = true                  # Email de confir
double_confirm_changes = true
secure_email_change_enabled = true

[auth.password]
min_length = 12
```

Extraction d'identité depuis le JWT — règle absolue

```typescript
// ❌ DANGEREUX — le user_id vient du body (contrôlé par l'
const { userId } = await req.json();

// ✅ Sécurisé — le user_id est extrait du JWT signé par Su
const { data: { user } } = await supabase.auth.getUser(jwtF
const userId = user.id;
```

Protection brute-force additionnelle

sql

```sql
CREATE TABLE login_attempts (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  email TEXT NOT NULL,
  ip_address INET,
  attempted_at TIMESTAMPTZ DEFAULT NOW(),
  success BOOLEAN DEFAULT FALSE
);

CREATE INDEX idx_login_attempts_email_time
ON login_attempts(email, attempted_at DESC);

CREATE OR REPLACE FUNCTION check_brute_force(p_email TEXT,
RETURNS BOOLEAN
LANGUAGE sql
SECURITY DEFINER
AS $$
  SELECT COUNT(*) < 5
  FROM login_attempts
  WHERE email = p_email
  AND success = FALSE
  AND attempted_at > NOW() - INTERVAL '15 minutes'
  AND (email = p_email OR ip_address = p_ip);
$$;
```

🏢 Sécurité multi-tenant

sql

```sql
-- Fonction helper d'isolation par company
CREATE OR REPLACE FUNCTION user_owns_company(company_id UUI
RETURNS BOOLEAN
LANGUAGE sql
SECURITY INVOKER
STABLE
AS $$
  SELECT EXISTS (
    SELECT 1 FROM companies
    WHERE id = company_id
    AND user_id = auth.uid()
  );
$$;

-- Policy lisible
CREATE POLICY "clients_belong_to_owned_company"
ON clients FOR ALL
USING (user_owns_company(company_id))
WITH CHECK (user_owns_company(company_id));
```

Test d'isolation à intégrer dans la CI

```typescript
```

```typescript
// tests/security/tenant-isolation.test.ts
describe('Tenant Isolation', () => {
  it('should not return invoices from another user', async
    const user1Client = createSupabaseClient(user1Token);
    const user2Client = createSupabaseClient(user2Token);

    const { data: invoice } = await user1Client
      .from('invoices')
      .insert({ amount: 1000, company_id: user1CompanyId })
      .select().single();

    // User2 essaie de lire la facture de User1
    const { data } = await user2Client
      .from('invoices')
      .select()
      .eq('id', invoice.id)
      .single();

    // RLS filtre silencieusement
    expect(data).toBeNull();
  });
});
```

📋 RGPD — Conformité

Ce qui est légalement obligatoire avant beta publique

```typescript
```

```
// Enregistrement du consentement
interface ConsentRecord {
  userId: string;
  version: string;       // version de la privacy policy
  consentedAt: Date;
  ipAddress: string;     // pour preuve légale
  userAgent: string;
}

// Droit à l'effacement — avec nuance légale pour facturati
async function handleDeletionRequest(userId: string) {
  // Les factures NE PEUVENT PAS être supprimées (obligatio
  // On anonymise les données personnelles, on conserve les
  await supabase.rpc('anonymize_user_data', { p_user_id: us
}
```

sql

```
-- Fonction d'anonymisation RGPD
CREATE OR REPLACE FUNCTION anonymize_user_data(p_user_id UU
RETURNS VOID
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
BEGIN
  UPDATE profiles
  SET name = 'Utilisateur supprimé', phone = NULL, address
  WHERE user_id = p_user_id;

  UPDATE clients
  SET
    name = 'Client anonymisé',
    email = NULL,
    phone = NULL,
    address = NULL,
    vat_number = NULL
  WHERE company_id IN (
    SELECT id FROM companies WHERE user_id = p_user_id
  );

  INSERT INTO gdpr_logs (user_id, action, executed_at)
  VALUES (p_user_id, 'anonymization', NOW());

  -- NE PAS supprimer : invoices, invoice_lines, amounts (o
END;
$$;
```

Politique de rétention automatique

```sql
CREATE OR REPLACE FUNCTION purge_expired_data()
RETURNS VOID
LANGUAGE sql
AS $$
  DELETE FROM rate_limit_logs WHERE created_at < NOW() - IN

  DELETE FROM audit_logs
  WHERE created_at < NOW() - INTERVAL '2 years'
  AND action NOT IN ('invoice_created', 'invoice_sent', 'pa
$$;

-- Planifier via pg_cron
SELECT cron.schedule('purge-expired-data', '0 2 * * 0', 'SE
```

📝 Audit Trail

```sql
```

```sql
CREATE TABLE audit_logs (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID REFERENCES auth.users(id),
  action TEXT NOT NULL,
  resource_type TEXT NOT NULL,
  resource_id UUID,
  metadata JSONB DEFAULT '{}',
  ip_address INET,
  user_agent TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_audit_user_time ON audit_logs(user_id, cre
CREATE INDEX idx_audit_resource ON audit_logs(resource_type

ALTER TABLE audit_logs ENABLE ROW LEVEL SECURITY;
CREATE POLICY "read_own_logs" ON audit_logs FOR SELECT USIN
```

typescript

```
// Middleware d'audit
export async function logAction(
  serviceSupabase: SupabaseClient,
  userId: string,
  action: string,
  resourceType: string,
  resourceId: string,
  metadata: Record<string, unknown>,
  req: Request
) {
  const ip = req.headers.get('x-forwarded-for')?.split(',')
  await serviceSupabase.from('audit_logs').insert({
    user_id: userId,
    action,            // 'invoice.created', 'invoice.sent',
    resource_type: resourceType,
    resource_id: resourceId,
    metadata,
    ip_address: ip,
    user_agent: req.headers.get('user-agent'),
  });
}

// Usage
await logAction(serviceClient, user.id, 'invoice.sent', 'in
  recipient: client.email,
  amount: invoice.total,
}, req);
```

🚨 Protection contre les abus

typescript

```typescript
// Détection bot
export function detectBot(req: Request): boolean {
  const ua = req.headers.get('user-agent') ?? '';
  const botPatterns = [
    /bot/i, /crawler/i, /spider/i, /scraper/i,
    /curl/i, /wget/i, /python-requests/i, /axios/i
  ];
  return botPatterns.some(p => p.test(ua));
}
```

tsx

```tsx
// Honeypot field côté frontend (invisible aux humains, rem
<input
  name="website"
  style={{ display: 'none' }}
  tabIndex={-1}
  autoComplete="off"
/>
```

typescript

```typescript
// Côté Edge Function
if (body.website && body.website.length > 0) {
  // Réponse fausse — on ne confirme pas la détection
  return new Response(JSON.stringify({ success: true }), {
}
```

4️⃣ Checklist sécurité SaaS complète

🔴 Bloquant — avant toute mise en production

- [ ] RLS activé et testé sur **toutes** les tables
- [ ] Aucune clé secrète dans le code frontend (`VITE_` prefix = public)
- [ ] Toutes les Edge Functions vérifient le JWT + `user_id` de la ressource
- [ ] Bucket Storage en mode privé, accès via URL signée uniquement
- [ ] Validation des uploads par magic bytes
- [ ] Headers de sécurité configurés dans `vercel.json`
- [ ] Email de confirmation activé à l'inscription
- [ ] Tests d'isolation multi-tenant dans la CI

🟠 Haute priorité — semaine 1-2

- [ ] Rate limiting sur auth endpoints et Claude API
- [ ] Audit trail sur toutes les actions sensibles
- [ ] Politique de confidentialité + DPA RGPD publiée
- [ ] SPF / DKIM / DMARC configurés pour le domaine email
- [ ] CSP header strict
- [ ] Rotation des refresh tokens activée
- [ ] Sanitisation DOMPurify si HTML affiché

🟡 Priorité normale — semaine 3-4

- [ ] `pg_cron` pour purge automatique des données expirées
- [ ] Honeypot + détection bot sur les formulaires publics
- [ ] Tests de pénétration basiques (OWASP ZAP ou Burp Suite)
- [ ] Procédure de réponse à incident documentée

- [ ] Monitoring d'erreurs (Sentry) avec données sensibles filtrées
- [ ] Protection SSRF avec whitelist domaines

🔵 Bonne pratique — post-MVP

- [ ] 2FA optionnel pour les utilisateurs
- [ ] Notifications de connexion par email
- [ ] Session management : voir ses sessions actives et les révoquer
- [ ] Rapport RGPD exportable (droit d'accès)
- [ ] Intégration CVE monitoring pour les dépendances (`npm audit` en CI)
- [ ] Bug bounty program

5️⃣ Recommandations prioritaires — Ordre d'implémentation

Semaine 1 — Éliminer les risques critiques

1. Activer RLS sur toutes les tables + écrire les policies + tests d'isolation
2. Auditer toutes les variables `VITE_` et déplacer les secrets en Edge Functions
3. Vérifier chaque Edge Function : extraction JWT + vérification `user_id` sur chaque ressource
4. Passer le bucket Storage en privé + URL signées

Semaine 2 — Solidifier l'architecture

5. Configurer les headers de sécurité dans `vercel.json`

6. Implémenter le rate limiting sur les endpoints sensibles
7. Créer la table `audit_logs` + logger les actions clés
8. Validation magic bytes sur les uploads

Semaine 3 — Conformité et protection

9. Rédiger Privacy Policy + DPA + page de consentement
10. Implémenter la fonction d'anonymisation RGPD
11. Configurer SPF/DKIM/DMARC sur le domaine
12. Activer `pg_cron` pour la purge des données

Semaine 4 — Durcissement final

13. Tests de pénétration avec OWASP ZAP
14. Honeypot + détection bot
15. Intégrer `npm audit` dans la CI/CD Vercel
16. Documenter la procédure de réponse à incident

> **Action immédiate** — Lance ce SQL dans ta console Supabase pour identifier les tables sans RLS :
>
> ```sql
> SELECT tablename FROM pg_tables
> WHERE schemaname = 'public'
> AND tablename NOT IN (SELECT tablename FROM pg_policies)
> ```

> Si cette requête retourne des lignes, tu as une faille critique ouverte en ce moment même.